Génération de musique jazz par IA

Effectué par

Spahr Archibald et Wu Thierry

Du 2/06/25 au 27/06/25

Lieu : Laboratoire d'Ingénierie des Systèmes de Versailles
10 Av. de l'Europe, 78140 Vélizy-Villacoublay
Nom du diplôme : L2LDDMP
Nom du tuteur : Topsu Suat et Guan Hongyu
Université de Versailles Saint-Quentin-en-Yvelines

Remerciements

Nous tenons à remercier tout d'abord nos deux tuteurs de stage : M. Suat Topsu et M. Guan Hongyu pour le partage de leurs connaissances sur le machine learning et en programmation.

Nous remercions Tuyet Touchais pour le matériel prêté et son aide pour configurer les GPU à distance.

Nous remercions également M. Benabou pour son aide avec les environnements virtuels.

Table des matières

1 Introduction **3**

2 Bibliographie **3**

2.1 L'architecture fondamentale du Transformer : ses différentes couches 3

2.1.1 La Couche d'Embedding 4

2.1.2 La Couche d'Attention 5

2.1.3 La Couche de Feedforward 8

2.2 Le mécanisme d'apprentissage 9

2.2.1 Le Mécanisme de Backpropagation et la Fonction de Perte 9

2.3 Les mécanismes additionnels qui permettent le bon fonctionnement du Transformer 10

2.3.1 La Normalisation de Couche (Layer Normalization) 10

2.3.2 Les Connexions Résiduelles (Skip Connections) 10

2.3.3 Le Dropout 11

3 Outils **12**

3.1 Outils et environnement de travail 12

3.2 Langage de programmation et bibliothèques installées 13

4 Méthode **13**

4.1 Etude Analytique 14

4.1.1 La Tokenization 14

4.1.2 Le squelette du Transformer 17

4.2 Etude Expérimentale 20

4.2.1 La programmation 20

4.2.2 Les problèmes rencontrés 20

5 Résultats et Analyse **21**

5.1 Résultat de l'entraînement 21

5.2 Résultat de génération musicale 22

5.3 Perspectives d'amélioration 23

6 Conclusion **23**

7 Annexe **25**

7.1 Dataset midi 25

1 Introduction

L'intelligence artificielle est devenue notre assistant au quotidien, elle répond à nos questions, résout des problèmes et aide à la prise de décisions. Cependant, il existe encore des domaines dans lesquels l'IA rencontre des difficultés à performer. Celui qui s'inscrit au cœur de notre stage intitulé "Génération de musique JAZZ par intelligence artificielle" en est un. En effet, il existe de nombreux générateurs de musique libre accès en ligne mais la qualité de leur rendu laisse à désirer, et ce même pour les modèles plus récents et développés.

Empiriquement, on remarque qu'ils sont capables de produire une musique agréable à l'écoute cependant dès que la requête devient spécifique, la réponse laisse à désirer. Suno AI par exemple, l'un des modèles les plus avancés et disponible au public depuis le 20 décembre 2023, génère lorsqu'on demande du "Jazz" une mélodie standard de piano, sans que l'on parvienne à discerner du Jazz à proprement parler. Comment se fait-il que dans un domaine aussi populaire que la musique, personne ne parvienne à créer un modèle qui nous fasse oublier que le résultat est le fruit d'une machine ?

Ce n'est pourtant ni la recherche ni l'investissement qui manquent en la matière, on compte aujourd'hui près de 350 millions d'euros de financements dans 19 entreprises différentes à travers le monde (**?**). L'enjeu économique est immense, le marché de l'industrie musicale généré par l'IA devrait en effet croître jusqu'à atteindre les 38,71 milliards de dollars d'ici 2033 (**?**). Ainsi, notre stage s'inscrit dans un enjeu global, à la fois culturel, technologique et économique, pleinement d'actualité. Etant donné la complexité du sujet, une approche naturelle est de chercher à acquérir une compréhension fondamentale des algorithmes qui régissent ce domaine. On en vient à se poser la question : Comment code t-on une IA capable de générer de la musique ? Quels sont les défis que l'on peut rencontrer au cours de cette quête ?

Ainsi l'objectif de ce stage est de programmer des modèles de machine learning et de les adapter à la génération de musique. Enfin il est aussi intéressant de s'appuyer sur une partie pratique à la fois pour mieux comprendre quelles sont les difficultés derrière la programmation d'un tel modèle et pour s'initier à la programmation d'une intelligence artificielle.

2 Bibliographie

2.1 L'architecture fondamentale du Transformer : ses différentes couches

Un modèle architectural fait référence à la structure ou à la conception d'un réseau de neurones ou d'un système d'apprentissage automatique. Cela inclut la disposition des différentes couches, les types de couches utilisées, et la manière dont elles sont connectées les unes aux autres. Il existe de nombreux modèles architecturaux en intelligence artificielle. Le Multi-Layer Perceptron (MLP) est la toute première de ces architectures à avoir vu le jour dans les années 1960, c'est le plus basique des modèles de réseaux de neurones, mais il est néanmoins encore utilisé aujourd'hui pour sa simplicité.

Le plus récent modèle architectural est le Transformer, présenté pour la première fois dans un article de recherche intitulé "Attention Is All You Need" par Vaswani et al, en 2017 (**?**). Il est très vite devenu le modèle de référence grâce à son efficacité et sa rapidité d'exécution, c'est pour cela que nous l'avons choisi pour la présente étude. Le transformer est constitué de

plusieurs couches dont le nombre peut varier de 2 à celui que l'utilisateur souhaite dépendant de la profondeur de réseau qu'il cherche à atteindre.

2.1.1 La Couche d'Embedding

Le transformer reste avant tout un modèle régi par une simple machine, incapable de donner du sens au discours comme nous le faisons. Alors pour qu'il parvienne à organiser des liens sémantiques entre des mots, on doit lui fournir la structure adéquate à la création de ces liens. Pour ce faire, on se projette dans un espace vectoriel multidimensionnel vide. Au sein de cet espace on va insérer des token que l'on extrait de nos données d'entraînement et qu'on initialise aléatoirement. Plus simplement, on prend la plus petite unité d'information que le modèle est capable d'interpréter, dans un modèle de génération de langage ça serait un mot par exemple, et on l'insère dans un endroit aléatoirement choisi au sein de cet espace vectoriel. Ces mots eux auront été extraits des données sur lesquelles nous entraînerons notre modèle plus tard.

À ce moment-là on dispose d'un espace vectoriel dans lequel se trouvent des mots aléatoirement placés, on pourrait retrouver par exemple "chat" à côté de "xylophone". Or, dans tout espace vectoriel il existe une notion de distance, que l'on appelle la norme. Cette norme permet au modèle d'établir des connexions sémantiques entre les tokens/mots , de la même manière que notre cerveau établit des connexions neuronales (**?**). Plus la norme qui sépare deux mots est faible et plus ces derniers ont de chances d'avoir la même signification. Les connexions issues de cet espace sont en un premier temps démunies de sens, mais elles sont plus tard mises à jour par le biais d'un mécanisme que l'on introduira dans la suite du rapport, et lorsque le modèle atteint maturité son espace vectoriel est composé de multiples espaces qui regroupent les mots sémantiquement rapprochés. Par exemple, si on raisonne dans un espace vectoriel de dimension 5, on pourrait voir le mot chat associé au vecteur [2,0,9,4,5] et le mot chien associé au vecteur [2,0,8,4,4] . Ce processus là est appelé processus d'"embedding".

Coder cette partie là du transformer est une des parties les plus simples à réaliser puisqu'il nous suffit d'extraire les données avec lesquelles on va s'entraîner et d'associer à chaque unité d'information un vecteur de dimension égale à celle de notre espace vectoriel (1) . Il est important de remarquer que plus cet espace vectoriel est grand, plus notre réseau sera capable de créer des connexions subtiles.

En plus d'associer à chaque mot une signification on peut aussi y associer une position. En effet, le modèle de transformer traite les données de manière parallèle plutôt que séquentielle, ce qui est l'une des raisons pour lesquelles il est si efficace et rapide. Conséquemment, il n'a aucune notion d'ordre au sein d'une séquence d'information que nous lui fournissons, il interprète la phrase "J'ai mangé un poisson" de la même manière qu'il interprète "un mangé j'ai poisson", ce qui pose évidemment problème. C'est pour cette raison là que l'on introduit un encodage en plus, le positionnel. Il se produit juste après l'encodage sémantique et fonctionne pareillement, on associe à chaque unité d'information un vecteur de dimension égale à celle de notre espace vectoriel. Enfin, pour chaque unité d'information on additionne son vecteur sémantique et positionnel afin que le vecteur résultant contienne à la fois des informations sur la signification du token et sa position dans la séquence. Cela permet au modèle de prendre en compte à la fois le sens des mots et leur ordre dans la séquence lors du traitement des données.

	Embedding sémentique	Embedding positionnel	Résultante
"Chat"	[0.1, 0.3, 0.5, 0.2]	[0.0, 0.1, 0.0, 0.1]	[0.1, 1.3, 0.6, 1.2]
"Mignon"	[0.2, 0.4, 0.6, 0.3]	[0.5, 0.5, 0.5, 0.5]	[0.7, 0.9, 1.1, 0.5]

Table 1 – vecteur sémantique et positionnel de chat et mignon

La résultante des mots “chat” et “mignon” est proche car ils sont souvent associés dans le langage courant.

2.1.2 La Couche d’Attention

Cette couche est à l’origine de toute l’innovation qu’amène le Transformer et est au cœur de son fonctionnement. C’est elle qui permet au modèle de comprendre les relations contextuelles entre les différentes unités d’information (tokens) d’une séquence. Poursuivons notre exemple de génération textuelle. Mettons qu’on fournit à notre modèle la phrase : *“Le chat mange le poisson”*. S’il veut pouvoir générer le prochain token/mot de manière cohérente, il ne doit pas uniquement se concentrer sur le mot *“poisson”*, sinon il générera simplement un mot qui se trouve proche de *“poisson”* dans notre espace vectoriel, ce qui pourrait rendre la phrase incohérente. Ainsi, il est impératif qu’il prenne en compte *l’ensemble du contexte* de la phrase.

Ce mécanisme qui lui permet de faire cela est celui de l’attention. Il existe plusieurs types d’attention, mais commençons par étudier la plus simple d’entre elles : la self-attention. Le modèle associe à chaque token trois vecteurs : une *clé* K, une *requête* Q et une *valeur* V, de dimension arbitraire d_k. Ces vecteurs sont produits à partir des embeddings via multiplication avec des matrices de projection aléatoires apprises :

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

où X est la matrice des embeddings (taille $n \times d_{model}$), et $W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_k}$ sont les matrices de projection. Le vecteur clé représente ce que le token a à offrir, la requête ce qu’il cherche à comprendre, et la valeur l’information transportée.

Chaque token va ensuite comparer sa requête avec les clés de tous les autres tokens, à l’aide d’un *produit scalaire*, afin de mesurer la pertinence de chaque interaction. Le score d’attention brut entre la requête Q_i et une clé K_j est donné par :

$$\text{score}(i, j) = Q_i \cdot K_j^T$$

Pour éviter que ces produits scalaires ne deviennent trop grands, on les normalise par la racine carrée de la dimension :

$$\text{score}(i, j) = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}}$$

Ensuite, on applique une fonction *softmax* sur chaque ligne afin de transformer ces scores en poids d’attention, formant une matrice $A \in \mathbb{R}^{n \times n}$, où chaque coefficient A_{ij} indique à quel point le token i doit prêter attention au token j.

Enfin, pour calculer la représentation contextuelle de chaque token, on combine ces poids avec les vecteurs valeurs correspondants :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Dans les réseaux de neurones, la fonction softmax est utilisée pour transformer un vecteur de scores en distribution de probabilité normalisée (**?**). Cela signifie que chaque valeur est comprise entre 0 et 1 et la somme totale des valeurs est égale à 1.

Si on prend un vecteur quelconque x = [x1, x2, ... , xn], la softmax est définie par : softmax(xn) = $\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$

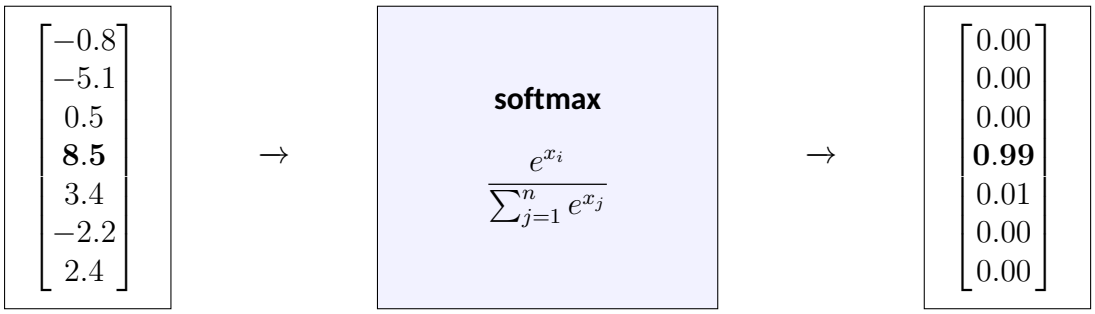


Figure 1 – Illustration de la fonction softmax : transformation d'un vecteur brut en distribution de probabilité.

Le résultat est une nouvelle matrice de vecteurs, enrichis non seulement de leur propre information mais aussi de celle des autres tokens qu'ils ont jugés importants. C'est ainsi qu'un Transformer est capable de prendre en compte l'ensemble du contexte d'une phrase avant d'en générer la suite.

2.1.2.1 Le masque d'attention

Il est important de toujours garder en tête l'objectif de son modèle lorsqu'on le construit. En effet, si l'on cherche à coder un modèle qui génère le sentiment d'une phrase , alors il est pertinent qu'il puisse accéder à l'intégralité des séquences (**?**). En revanche, si l'on souhaite construire un modèle qui génère des phrases mot par mot, à la manière de ChatGPT, il devient crucial que le modèle ne puisse pas voir les tokens futurs, sans quoi la phase d'apprentissage serait biaisée et le modèle mal calibré lors de l'inférence.

Supposons par exemple que le modèle est en train de générer la phrase : « Le chat dort. » . À l'étape 2, lorsqu'il génère le mot « chat » , il ne doit pas voir le mot « dort » . Mais dans un Transformer, l'attention est naturellement globale, chaque token peut observer tous les autres. Il faut donc restreindre cette attention en masquant les tokens futurs, pour que le modèle n'ait accès qu'aux mots qu'il a déjà vus ou qu'il est en train de traiter.

Pour cela, on applique un masque à la matrice des poids d'attention (**?**). Ce masque prend la forme d'une matrice triangulaire inférieure : chaque token ne peut porter attention qu'aux tokens situés avant lui dans la séquence (ou à lui-même). Le résultat est une matrice où les valeurs au-dessus de la diagonale sont annulées avant le softmax, empêchant le modèle de regarder en avant.

$$A = \begin{bmatrix} 1.0 & 0 & 0 & 0 \\ 0.6 & 0.4 & 0 & 0 \\ 0.3 & 0.4 & 0.3 & 0 \\ 0.2 & 0.3 & 0.3 & 0.2 \end{bmatrix} \tag{1}$$

Note : L'élément $A_{3,1} = 0.3$ représente l'attention que le mot « dort » (position 3) porte au mot « Le » (position 1).

2.1.2.2 L'attention à plusieurs têtes dite Multi-Head Attention

Le problème avec l'attention dans sa forme la plus simple, est qu'il y a une seule matrice de poids pour toutes les clés et requêtes, donc lorsque l'on souhaite mettre à jour notre matrice de poids et disons augmenter un poids afin d'avoir une meilleure affinité entre deux vecteurs, cela pourrait influencer une autre affinité, ce qui nous ferait perdre en précision. (**?**)

Afin de limiter cela, on introduit l'attention à plusieurs têtes, chaque tête se comporte comme ce que l'on vient de décrire, mais différentes têtes se spécialisent dans différentes relations. Par exemple, lorsque l'on construit un modèle de génération de langage, une tête va se concentrer sur la relation syntaxique dans les phrases tandis qu'une autre va se focaliser sur les relation rythmiques dans une phrase. L'output de chacune des têtes est ensuite concaténé pour obtenir un vecteur qui contient les informations contextuelles de chaque domaines dans lesquels les têtes se sont spécialisées (syntaxique,rythmique,etc..).

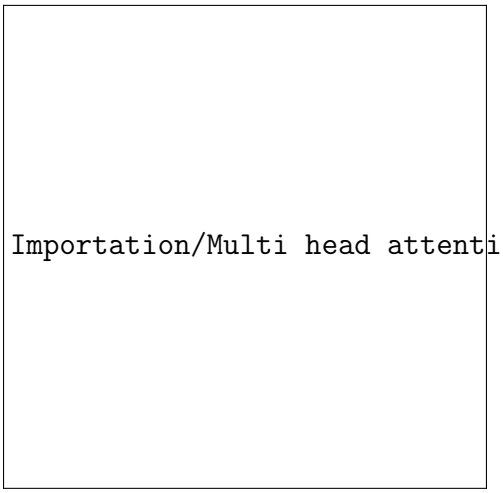


Figure 2 – Multi head attention

Il est important de noter que c'est impossible de véritablement choisir soi-même le domaine de spécialisation d'une tête, cependant l'initialisation aléatoire des matrices de clé requête et valeur dans chaque tête assure le fait qu'elles se spécialiseront quasiment systématiquement dans un domaine différent. Il peut toutefois arriver que deux têtes se spécialisent dans le même domaine et que donc l'une d'entre elles devienne inutile et entrave l'efficacité de notre modèle en lui infligeant des calculs inutiles, dans ce cas il suffit de supprimer cette tête. Cela se fait très facilement car les têtes sont indépendantes entre elles.

2.1.2.3 La Cross Attention

La *cross-attention* est un mécanisme clé dans les architectures Transformer à plusieurs flux, notamment dans les modèles qui gèrent plusieurs modalités simultanément (comme le texte et la musique, ou différents aspects d'un événement musical) (**?**). Contrairement à la self-attention, où les tokens d'une même séquence se comparent entre eux, la cross-attention permet à la requête $\mathbf{Q}$ d'une séquence cible d'interroger les clés $\mathbf{K}$ et les valeurs $\mathbf{V}$ d'une séquence source.

Ce mécanisme permet par exemple à la représentation d'un pitch de s'ajuster en fonction de ses dimensions temporelles associées (barre, décalage, durée), en extrayant l'information contextuelle utile depuis d'autres représentations. Le calcul de la matrice d'attention suit toujours la même logique : on applique un produit scalaire entre les requêtes $\mathbf{Q}$ et les clés $\mathbf{K}$, que l'on normalise et pondère via un softmax. La formule est la suivante :

$$\text{Attention}(\mathbf{Q}_{\text{cible}}, \mathbf{K}_{\text{source}}, \mathbf{V}_{\text{source}}) = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{cible}} \cdot \mathbf{K}_{\text{source}}^{\top}}{\sqrt{d_k}}\right) \cdot \mathbf{V}_{\text{source}}$$

où d_k est la dimension des clés. Ce mécanisme permet à chaque vecteur de la séquence cible de pondérer dynamiquement les informations qu'il extrait depuis la séquence source, selon leur pertinence contextuelle. Ainsi, il y à une fusion sémantique entre différents types d'informations, ce qui renforce la capacité du modèle à produire des séquences cohérentes et expressives.

2.1.3 La Couche de Feedforward

La couche de feedforward vient juste après la couche d'attention, agit comme un traitement individuel appliqué à chaque token, indépendamment de son contexte. En effet, une fois que chaque token a été enrichi par l'attention avec des informations issues des autres tokens, il est nécessaire de lui faire subir une transformation plus profonde et non-linéaire, qui va lui permettre de moduler et d'affiner sa propre représentation interne.

Mathématiquement, cette couche est composée de deux couches linéaires séparées par une fonction d'activation non-linéaire. Si l'on note $x \in \mathbb{R}^{d_{\text{model}}}$ le vecteur d'entrée pour un token donné, la transformation effectuée par la couche de feedforward est :

$$\text{FFN}(x) = W_2 \cdot \phi(W_1 \cdot x + b_1) + b_2$$

où :
- $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ et $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ sont les matrices de poids,
- $b_1 \in \mathbb{R}^{d_{\text{ff}}}$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$ sont les biais associés,
- ϕ est une fonction d'activation non-linéaire, généralement ReLU :

$$\phi(z) = \max(0, z)$$

ou GELU :

$$\phi(z) = z \cdot \Phi(z) \quad \text{où } \Phi(z) = \frac{1}{2}\left(1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right)$$

Lors du passage dans cette couche de feedforward , chaque vecteur est d'abord projeté dans un espace de plus grande dimension ($d_{\text{ff}} > d_{\text{model}}$), ce qui permet au modèle de recombiner les différentes composantes du vecteur pour en extraire des caractéristiques plus abstraites. Ensuite, une seconde projection ramène ce vecteur à sa dimension d'origine. Le rôle de cette

couche est donc de transformer les représentations produites par l'attention en des représentations plus discriminantes et utiles pour la suite du traitement.

Bien que cette opération soit appliquée séparément à chaque token, elle est cruciale car elle permet au modèle de capturer des motifs complexes, de filtrer les informations inutiles, et d'enrichir les représentations intermédiaires avec des comportements non-linéaires que la simple attention ne saurait modéliser seule.

2.2 Le mécanisme d'apprentissage

2.2.1 Le Mécanisme de Backpropagation et la Fonction de Perte

Le cœur de l'apprentissage dans un modèle Transformer repose sur un processus appelé backpropagation, ou rétropropagation du gradient. Ce mécanisme permet au modèle d'ajuster ses paramètres internes — tels que les poids des matrices d'attention ou de feedforward, afin de produire des prédictions de plus en plus précises au fil de l'entraînement. Pour savoir dans quelle mesure une prédiction est correcte ou non, on utilise une fonction de perte, typiquement la *Negative Log-Likelihood* (NLL), qui mesure l'écart entre la prédiction du modèle et la vérité attendue. (**?**)

Supposons qu'à une étape donnée, le modèle doive prédire le mot suivant dans une séquence, et qu'il produise une distribution de probabilités sur le vocabulaire :

$$\hat{y} = [P_1, P_2, \ldots, P_k]$$

où P_i est la probabilité prédite pour le i-ième mot du vocabulaire. Si le mot correct est le mot w_j, alors la perte NLL est définie par :

$$\mathcal{L} = -\log(P_j)$$

Cette valeur est d'autant plus faible que P_j est proche de 1 (une probabilité de 1 singifie que le modèle est 100 % sur). Une fois cette erreur mesurée, la rétropropagation entre en jeu. Grâce à la *dérivation automatique*, on calcule pour chaque paramètre θ du modèle le gradient de la perte :

$$\frac{\partial \mathcal{L}}{\partial \theta}$$

Ce gradient indique dans quelle direction et avec quelle amplitude modifier le paramètre pour réduire la perte lors du prochain passage. Ces gradients sont ensuite utilisés dans une mise à jour via une règle d'optimisation, par exemple avec l'algorithme **Adam** :

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

où η est le taux d'apprentissage. Ce processus est répété à chaque batch, sur des milliers voire des millions de séquences, ce qui permet au modèle de converger vers un état où il maîtrise la structure statistique de la langue ou de la musique. Ainsi, même si le modèle est une simple machine sans intuition, ce mécanisme mathématique d'erreur corrigée par rétropropagation lui permet d'apprendre à prédire des séquences de manière fluide, logique et cohérente.

2.3 Les mécanismes additionnels qui permettent le bon fonctionnement du Transformer

2.3.1 La Normalisation de Couche (Layer Normalization)

Dans les réseaux de neurones profonds, les activations peuvent rapidement devenir instables, soit trop grandes, soit trop petites, ce qui complique grandement l'entraînement. Pour pallier ce problème, le Transformer introduit un mécanisme de régulation appelé Layer Normalization (**?**). Contrairement à la Batch Normalization, qui calcule la moyenne et l'écart-type d'un neurone sur l'ensemble des exemples d'un batch, la *LayerNorm* agit au niveau d'un seul token, et normalise toutes ses dimensions individuellement, en se basant uniquement sur ses propres composantes.

Concrètement, pour un vecteur d'entré $x = [x_1, x_2, \ldots, x_d] \in \mathbb{R}^d$, représentant un token avec d dimensions (par exemple $d = 512$), la normalisation se fait comme suit :

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \quad \text{et} \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 + \varepsilon}$$

où μ est la moyenne du vecteur, σ son écart-type, et ε une petite constante pour éviter la division par zéro. Ensuite, chaque composante du vecteur est recentrée et redimensionnée :

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}$$

À cette normalisation, on applique deux vecteurs appris : un vecteur d'échelle $\gamma \in \mathbb{R}^d$ et un vecteur de biais $\beta \in \mathbb{R}^d$. Le vecteur final devient alors :

$$\text{LayerNorm}(x)_i = \gamma_i \cdot \hat{x}_i + \beta_i$$

Cette opération est généralement appliquée avant ou après les sous-couches du Transformer (attention et feedforward), et couplée à une connexion résiduelle. Elle joue un rôle fondamental dans la stabilité de l'entraînement du Transformer (**?**), en assurant que les vecteurs traités gardent une échelle cohérente à chaque couche, ce qui améliore la propagation des gradients et accélère la convergence. Si les valeurs de notre modèle sont écrasées alors le LayerNorm leur redonnera de l'amplitude et inversement si elles sont très larges il les réduira. Sans cette normalisation, les réseaux profonds comme les Transformers auraient beaucoup plus de mal à apprendre efficacement, surtout sur des longues séquences ou de grandes architectures.

2.3.2 Les Connexions Résiduelles (Skip Connections)

Les connexions résiduelles sont un autre élément fondamental du Transformer, bien qu'elles soient relativement simples dans leur principe. Lorsqu'un vecteur passe à travers une sous-couche (par exemple la couche d'attention ou la couche feedforward), le résultat est ensuite additionné au vecteur d'entrée initial avant d'être envoyé à l'étape suivante. Autrement dit, le modèle ne transmet pas uniquement le résultat transformé, mais l'ajoute à ce qu'il avait déjà reçu en entrée. Ce mécanisme, appelé connexion résiduelle, permet au modèle de conserver une trace de l'information brute à chaque étape du traitement, et de faciliter l'apprentissage de modifications progressives plutôt que de transformations radicales.

Mathématiquement, pour une entrée x et une sous-couche F, la sortie est donnée par :

$$\text{Sortie} = x + F(x)$$

Pendant l'apprentissage, le gradient se propage selon :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \text{Sortie}} \cdot \left(1 + \frac{\partial F(x)}{\partial x}\right)$$

D'un point de vue mathématique, cela permet de lutter contre la disparition ou l'explosion des gradients pendant l'entraînement, ce qui est particulièrement crucial dans les architectures profondes comme les Transformers qui peuvent compter plusieurs dizaines voire centaines de couches. En pratique, cela veut dire que si une sous-couche ne parvient pas à apprendre quelque chose d'utile (au début de l'entraînement, par exemple), le modèle peut toujours "passer l'information telle quelle" à la couche suivante. C'est pour cette raison que les connexions résiduelles sont quasiment toujours combinées avec une normalisation de couche : on applique souvent un schéma du type "entrée + transformation, puis normalisation", afin d'assurer à la fois la stabilité et la fluidité du passage de l'information.

2.3.3 Le Dropout

Le dropout est une technique introduite pour réduire le sur-apprentissage (ou overfitting) lors de l'entraînement des réseaux de neurones profonds, y compris les Transformers. Le principe est simple : à chaque étape d'entraînement, on va désactiver aléatoirement un certain pourcentage des neurones dans une couche donnée, ce qui empêche le réseau de devenir trop dépendant d'un sous-ensemble de neurones spécifiques. (**?**)

Techniquement, pour une activation h et un taux de dropout p, on crée un masque aléatoire m où chaque valeur a une probabilité $1 - p$ d'être conservée. La probabilité qu'une valeur soit mise à 0 est calculé par :

$$h_{\text{dropout}} = m \odot h$$

où $\odot$ représente le produit élément par élément.

À l'inférence, on applique un scaling pour conserver l'espérance :

$$h_{\text{inférence}} = (1 - p) \cdot h$$

Dans le cas du Transformer, cette régularisation est appliquée à plusieurs endroits stratégiques du bloc : à la sortie de l'attention multi-tête, à la sortie du feedforward, ainsi qu'après les embeddings positionnels et de tokens. En pratique, cela signifie que certaines dimensions des vecteurs sont mises à zéro de manière temporaire, uniquement pendant l'entraînement. Cela force le réseau à distribuer l'information de manière plus robuste et redondante, ce qui améliore sa généralisation à des données jamais vues.

Une fois l'entraînement terminé, ce masquage aléatoire est désactivé et le réseau utilise tous ses neurones lors de l'inférence. Bien que le dropout puisse sembler contre-intuitif - masquer volontairement des informations - il s'agit en réalité d'une méthode très efficace pour éviter qu'un modèle ne mémorise excessivement les données d'entraînement, et donc pour améliorer sa capacité à produire des résultats fiables sur des entrées nouvelles. (**?**)

Ainsi, au cours des deux premières semaines de ce stage nous nous sommes documentés et avons appris comment fonctionnait un transformer, l'architecture que l'on retrouve aujourd'hui

dans les modèles les plus pointus de machine learning. Après cela nous sommes passés à la phase plus expérimentale de notre stage ou nous avons programmé une IA capable de générer de la musique, mais passer de la théorie à la pratique et notamment en informatique, peut très souvent s'avérer délicat.

3 Outils

3.1 Outils et environnement de travail

L'ensemble des expérimentations a été réalisé sur un serveur distant mis à disposition par le laboratoire de LISV, disposant d'une configuration matérielle puissante, notamment équipée d'un GPU dédié de dernière génération (modèle NVIDIA) et de coeur CUDA, il s'agit d'une interface utilisateur permettant d'exécuter des fonctions sur un processeur nvidia. Elle est utile pour l'informatique parallèle, qui consiste à effectuer simultanément de nombreux calculs indépendamment. Il est alors adapté aux charges de travail liées à l'intelligence artificielle, et en particulier à la génération musicale.

L'accès au serveur s'est effectué via une connexion SSH à l'aide du logiciel PuTTY, un client de terminal permettant de prendre la main sur une machine distante en ligne de commande. L'utilisation de WinSCP est également recommandé pour pouvoir transférer des fichiers vers le serveur distant et vice-versa. Ce mode de travail permet d'exécuter des scripts Python, d'installer des bibliothèques et de gérer les fichiers à distance, tout en utilisant les ressources matérielles du serveur. Ce schéma ci-dessous résume comment fonctionne une connexion SSH et comment on a travaillé. La connexion SSH sera dévellopé dans l'annexe.

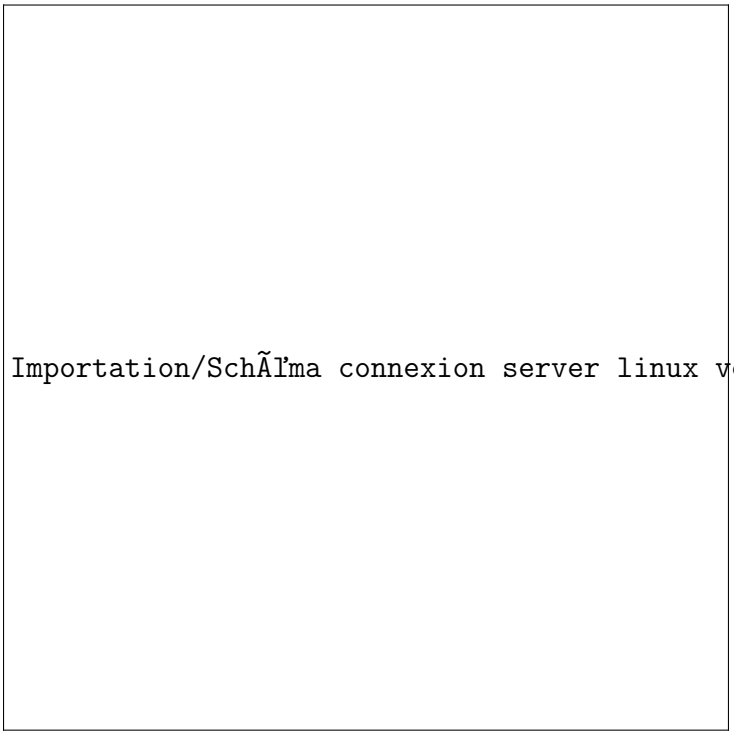


Figure 3 – Schéma de la connexion SSH

En complément de Putty, l'éditeur Visual Studio Code (VS Code) a été utilisé pour le développement des scripts Python sur un notebook Jupyter. L'utilisation du notebook a également

nécessité l'installation d'un environnement virtuel développé dans l'annexe.

Le serveur distant tournait sous une distribution Linux (Ubuntu). L'environnement de développement reposait sur Python 3.12, avec la gestion des dépendances assurée via pip. Les scripts ont été lancés depuis le terminal ou VS Code, et les fichiers générés (MIDI, audio, logs) ont été transférés ponctuellement vers une machine locale pour être analysés et écoutés.

Enfin, des outils comme Audacity ont été utilisés pour visualiser les fichiers MIDI et ont permis d'écouter les sorties audio et de valider la qualité musicale des échantillons générés.

3.2 Langage de programmation et bibliothèques installées

L'ensemble des projets a été développé en Python, un langage particulièrement adapté à la recherche et au prototypage en intelligence artificielle grâce à son nombre de bibliothèques spécialisées.

Les bibliothèques que nous avons utilisées sont les suivantes :

— PyTorch : C'est une bibliothèque conçue pour le calcul numérique basé sur des tensors, un tenseur étant une structure de données fondamentale et est largement utilisée pour construire et entraîner des réseaux de neurones profonds. C'est une bibliothèque très facile d'utilisation pour les débutants comme nous car elle généralise de nombreuses opérations courantes et complexes dans des réseaux de neurones en fonctions nominatives simplifiées. Lien vers la librarie

— Miditok : C'est une bibliothèque Python open source qui permet de convertir des fichiers MIDI en séquences de tokens, pour ensuite les utiliser afin d'entraîner son modèle.(**?**)

— Pretty_midi : C'est une bibliothèque Python qui sert à lire, écrire, manipuler et analyser des fichiers MIDI. Elle permet d'extraire facilement des attributs tels que les notes, instruments, tempos, dynamiques, etc. Lien vers la librarie

L'installation de ces outils s'est faite via pip, conda, ou via des scripts git clone et l'utilisation du terminal. Certains modèles ont besoin d'environnements spécifiques ou des versions précises de bibliothèques (par exemple, la version torch == 2.1.0 pour utiliser Audiocraft), ce qui a nécessité la création de plusieurs environnements pour chaque modèle utilisé.

En nous appuyant sur ces outils, nous avons ensuite débuté les études anlytiques et expérimentales.

4 Méthode

Une fois au point sur l'architecture du transformer, nous nous sommes penchés sur la programmation d'un modèle de transformer qui puisse générer de la musique.

4.1 Etude Analytique

4.1.1 La Tokenization

La première chose à faire lorsque l'on aborde la conception d'un modèle de Transformer, est de se pencher sur la manière dont on veut tokeniser le sujet d'étude. En clair, on cherche à comprendre comment récupérer et fournir une unité d'information à notre modèle dans le format le plus optimal à l'analyse de cette information. Dans le cadre de notre stage, le sujet d'étude est la musique, et plus précisément le jazz. Avant toute chose il faut trouver un moyen de récupérer des séquences de jazz et les mettre sous un format qui soit exploitable par un transformer.

Il existe aujourd'hui deux façons de tokenizer des morceaux de musique, la première méthode est la "Compound method" et la deuxième est la "Time-based method".

— **Compound Method** : Chaque unité d'information / token est une note de musique contenant l'information sur cette note comme le ton, la durée, le volume et plus encore en fonction de la précision de notre modèle. La temporalité est traitée à l'aide de "TimeStamps", des timbres de temporalité incarnés dans chaque token qui indiquent le moment du début de la note. Cela suffit au traitement de la temporalité puisqu'on trouve aussi dans le token la durée de la note, ainsi le modèle peut par un calcul simple déterminer le moment où se termine la note. La simultanéité, une notion primordiale si l'on souhaite générer des polyphonies et non seulement des monophonies, est traitée toujours avec le TimeStamp. Deux notes que l'on veut lire au même moment ont donc simplement le même TimeStamp.

— **Time-based method** : Chaque unité d'information est un attribut d'une note, deux tokens peuvent représenter deux choses différentes comme le ton et le volume de la note. La temporalité est traitée à l'aide de tokens "TimeShift" qui signalent une avancée temporelle, le TimeShift 0 signale le début de la musique. La simultanéité est signalée par l'absence de token TimeShift, si deux notes ne sont pas séparées par un token TimeShift alors elles seront jouées en même temps.

4.1.1.1 Compound Tokenizer vs Time-Based Tokenizer

La Compound Method est plus intuitive et facile à manœuvrer car chaque token représente une note complète. En une seule unité, elle encode toutes les informations nécessaires à la génération : la hauteur (pitch), le moment de début (timestamp), la durée ou la vélocité. À l'inverse, la Time-Based Method est plus fragmentée : chaque attribut est représenté par un token indépendant (par exemple un token pour la hauteur, un autre pour le temps, etc.), ce qui complexifie la représentation mais offre une flexibilité considérable.

Méthode	Structure des tokens
Compound	`[pitch=60, time=120, velocity=80]`
Time-Based	`[pitch=60]` $\rightarrow$ `[bar=2]` $\rightarrow$ `[offset=24]` $\rightarrow$ `[duration=0.4]`

Cependant un enjeu crucial nous a obligés à opter pour la Time-Based Method : la taille du vocabulaire, ou `vocab_size`. La `vocab_size` correspond au nombre total de tokens distincts qu'un modèle peut générer.

Exemple simple : Dans un modèle de génération de texte lettre par lettre, on a :

$$\texttt{vocab_size} = 26$$

car les 26 lettres de l'alphabet constituent l'ensemble des possibilités à chaque étape de génération.

Dans le cas du Compound Tokenizer :
Chaque token encode plusieurs attributs. Supposons qu'un token contient :
— une hauteur : 128 valeurs possibles (0–127)
— un timestamp : 1000 subdivisions possibles
— une vélocité : 128 valeurs possibles
Alors le vocabulaire total devient :

$$\texttt{vocab_size} = 128 \times 1000 \times 128 = \textbf{16 384 000}$$

> Cette croissance exponentielle rend la méthode compound impraticable en termes de mémoire et de complexité avec nos ressources actuelles.

Ainsi, malgré son intuitivité, le Compound Tokenizer devient rapidement explosif. Le Time-Based Tokenizer, en répartissant l'information sur plusieurs tokens, permet de maintenir un `vocab_size` plus raisonnable, ce qui facilite l'entraînement et la généralisation du modèle. Bien qu'intuitivement on pense que la `vocab_size` est la même ce n'est en réalité pas le cas car la `vocab_size` correspond au nombre total de tokens distincts qu'un modèle peut générer et non au nombre de combinaisons possibles de tokens. Alors dans le Time-Based Tokenizer la croissance de la `vocab_size` est additive et non multiplicative.

Pour cette raison, nous avons opté pour la Time-based method, ou plutôt nous nous sommes inspirés de cette méthode afin de réaliser notre propre tokenization.

Notre Tokenization :

Nous avons cherché à trouver le meilleur compromis entre précision musicale et efficacité computationnelle tout en restant dans la mesure de nos moyens. En un premier temps nous avons limité le nombre de tokens que l'on utilise à 4 : le ton MIDI (pitch), le numéro de mesure (bar), le décalage en ticks à l'intérieur de la mesure (off) et la durée arrondie au dixième de temps (end). Cela représente donc 1 token pour la musicalité et 3 pour la temporalité. Ce choix n'est pas anodin puisque nous avions décidé de nous focaliser plus sur la temporalité que la musicalité afin de nous assurer de générer un morceau qui certes moins riche musicalement,ait de la cohérence temporellement.

Rangée	Nom du token	Domaine de valeurs	Sens musical	Rôle d'encodage (pour le modèle)
0	pitch	0-127 (MIDI)	Hauteur de la note.	Permet au modèle d'apprendre les successions mélodiques
1	bar	0-1023	Numéro de mesure (barre de mesure) depuis le début du fichier.	Indique à quel numéro de mesure (0, 1, 2, …) se situe chaque événement musical. C'est le macro-rythme
2	off	0-191	Décalage intra-mesure : chaque barre est découpée en 48 ticks (1 tick = 1/48 de battue).	Position locale à l'intérieur de la mesure ; le masque temporel se base dessus pour garantir la causalité (notes futures masquées).
3	end	0-999	Durée quantifiée au dixième de battue. token = durée × 10 (ex. 1,6 beat → 16).	Encode la longueur de la note.

Table 2 – Description des tokens musicaux et leur encodage

Pourquoi ce choix ?

4.1.1.2 L'efficacité

La division en bar + offset (**?**) nous semblait être très adéquate à nos contraintes informatiques car elle permet de garder une vocab_size raisonnablement petite comparée à une grille absolue (aucune subdivision de temps donc le nombre de tokens explose avec la durée du morceau) . Avec une grille absolue : chaque tick possède un identifiant unique : 0, 1, 2, jusqu'à 12287 pour un morceau de 64 mesures à 1/48 de battue par exemple. Avec le système bar+offset, bar ne monte que d'une unité toutes les 4 battues (en 4/4), donc sur ce même exemple on aurait 64 bars + 192 offset = 256 tokens, soit un facteur diminutif de 48.

4.1.1.3 Flexibilité

Comme le démontre (**?**), La résolution au sein même d'un bar est très flexible, il suffit de changer un seul paramètre et cela n'affecte pas le reste du code et peu la vocab_size. C'est très pratique lorsque l"on souhaite entraîner un modèle pas à pas avec une résolution de plus en plus précise

4.1.1.4 Rythme

Les tokens bar permettent au modèle de trouver des repères rythmiques plus aisément, ainsi on espère que notre modèle aura un meilleur sens du rythme que si on lui avait donné une grid absolue. **Comparaison des Méthodes**

Critères	Compound Method	Time-based Method	Notre méthode
Taille vocabulaire	16,112,871 (grand)	∼1,500 (minimaliste)	2,340 (optimisé)
Flexibilité	Rigide	Modulaire	Semi-modulaire
Adaptabilité	Faible	Élevée	Contrôlée
Personnalisation	Complexe (trop d'attributs fixes)	Simple	Ciblée (focus temporalité)
Complexité calcul	O(n) (mais gros vocabulaire)	O(n) (traitement parallèle facile)	O(n) + optimisations
Précision musicale	Maximale (tous détails préservés)	Variable (dépend des découpages)	Ciblée (temps > timbre)
Ressources requises	GPU haute mémoire (>32GB)	CPU/GPU standard	GPU standard accessible (8GB)

Table 3 – Comparaison des différentes méthodes

4.1.2 Le squelette du Transformer

Une fois fini notre tokenization, il était temps de se pencher sur la structure de notre Transformer. Nous nous sommes appuyés sur le processur que nous avions établi pour la base puis nous avons introduit quelques spécificités pour notre objectif. Le choix de l'architecture du modèle s'est porté sur une variante personnalisée du Transformer, inspirée des modèles de type GPT, mais adaptée aux contraintes structurelles et temporelles propres à la musique. Plutôt que d'utiliser un Transformer classique ou une approche naïve de séquences, nous avons opté pour une architecture multi-stream conçue pour refléter fidèlement la richesse musicale et permettre un apprentissage plus efficace.

4.1.2.1 Une division en 4 flux : La spécialisation

L'architecture du modèle repose sur 4 flux parallèles, un par type de token (pitch, bar, offset, duration). Chaque flux est traité indépendamment par une pile de blocs Transformer, c'est à dire que l'on empile plusieurs fois le bloc de base introduit dans la bibliographie, avec une couche d'attention, une couche de FNN et des LayerNorm. Cela permet de donner de la profondeur au réseau de neurones ce qui permet au modèle d'apprendre des représentations complexes et spécialisées pour chaque dimension. Par exemple, le flux pitch apprend des régularités mélodiques, le flux bar capte la structure métrique (position dans le morceau),le flux offset modélise les placements rythmiques fins et le flux duration apprend des motifs de tenue de note. Cette séparation renforce la capacité d'abstraction du modèle et améliore son interprétabilité.

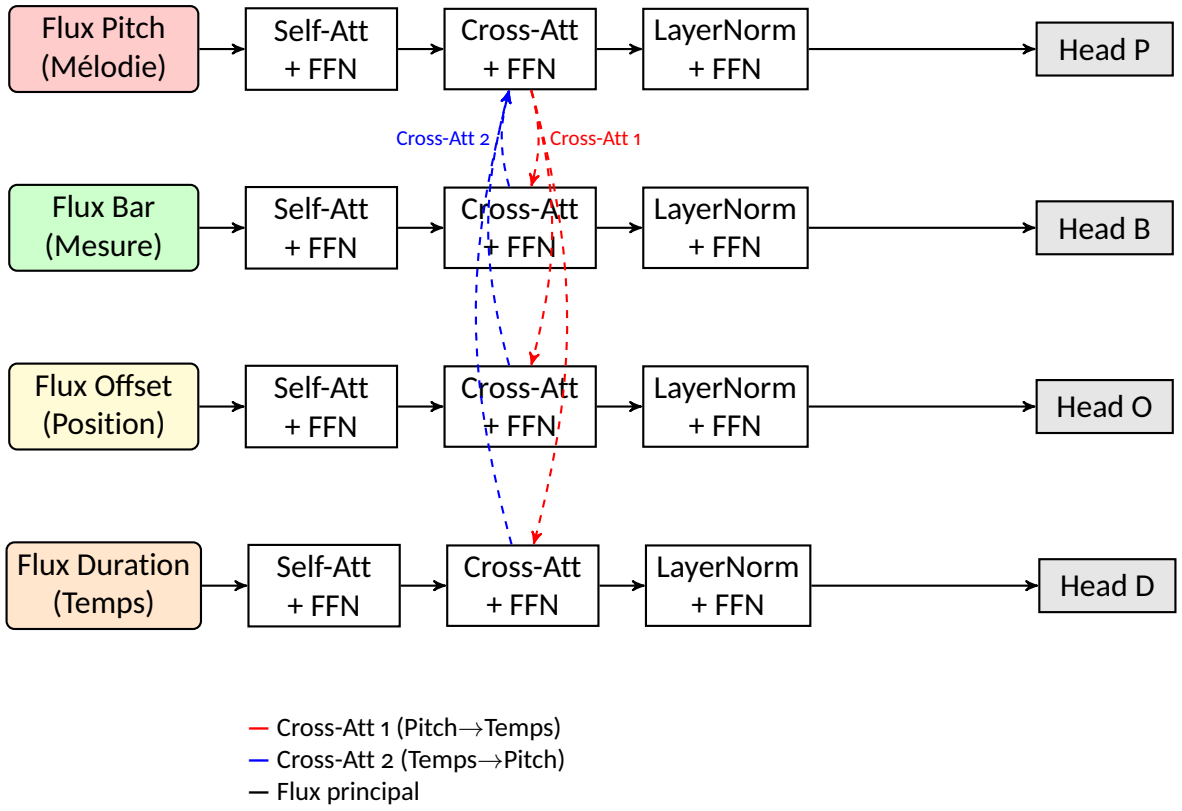


Figure 4 – Architecture avec interactions à la 2ème couche et sorties indépendantes

4.1.2.2 Transformater en 4 flux

On a ensuite inséré un mécanisme de cross-attention entre les 4 flux, afin de permettre au modèle de capturer des interdépendances explicites entre les différentes composantes d'un événement musical. Ce mécanisme est organisé en deux parties entre les attributs temporels et l'attribut musical. On calcule l'attention entre : les clés du pitch, et les requêtes + valeurs des bars, off et end. Puis l'attention entre les clés des token bars, off et end les requêtes + valeurs du pitch.

Une fois les valeurs recueillies, on ajoute un petit réseau FNN pour affiner le résultat. Ce mécanisme permet d'assurer une cohérence entre les différents aspects de la musique et d'établir des liens entre les différents tokens. Par exemple, certaines hauteurs de note (pitch) sont plus probables d'apparaître à certaines positions métriques (bar, offset).

Somme toute, on applique à chaque token une transformation à travers plusieurs blocs simples du transformer décrit dans la bibliographie, puis on mélange toutes ces informations recueilles dans un bloc de Cross Attention, et enfin on affine à nouveau ce résultat avec un simple FNN.

4.1.2.3 Quelques ajouts nécessaires

4.1.2.4 Embeddings de Position

Un Transformer traite les informations d'une même séquence en parallèle, grâce à des opérations matricielles avec poids partagés. C'est ce qui le rend si efficace, notamment sur les longues séquences. Cependant, cette parallélisation pose un problème : puisqu'il traite tous

les tokens simultanément, le modèle n'apprend aucune notion d'ordre par défaut. Il devient alors incapable de distinguer une séquence comme :

`[do, ré, mi]` vs `[mi, ré, do]`

alors que ces deux séquences ont des significations musicales très différentes.

Pour remédier à cela, on ajoute généralement des **embeddings de position absolus** : chaque token reçoit un vecteur qui encode sa position fixe dans la séquence. Cela permet au modèle d'apprendre la notion de position linéaire.

Position	Embedding Absolu
0	$\vec{p}_0$
1	$\vec{p}_1$
2	$\vec{p}_2$
⋮	⋮

Cependant, dans un contexte musical, cette approche atteint ses limites. En musique, ce sont les relations relatives qui priment — par exemple, savoir qu'une note suit immédiatement une autre, indépendamment de leur position exacte dans la partition. Un motif comme :

`[do, ré, mi]` à la mesure 2 $\rightsquigarrow$ `[do, ré, mi]` à la mesure 48

garde une structure identique. Or, avec des embeddings absolus, le modèle devrait réapprendre ce motif à chaque nouvelle position, ce qui nuit à la généralisation.

C'est pourquoi nous avons intégré des **embeddings de position relatifs** (**?**). Plutôt que d'encoder une position fixe, le modèle encode la distance entre les tokens, via une matrice d'embedding relationnel :

$$\boxed{\text{position relative} = j - i \qquad \Rightarrow \qquad \text{embedding} = \mathbf{r}_{j-i}}$$

Ces embeddings sont directement injectés dans le calcul des scores d'attention :

$$\text{score}_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j + \mathbf{q}_i \cdot \mathbf{r}_{j-i}$$

Ils permettent donc au modèle de comparer les tokens non pas par leur place dans la séquence, mais par leur **écart temporel réel** (`bar` et `offset`), ce qui permet une meilleure reconnaissance des motifs rythmiques et mélodiques. Ce mécanisme rend l'attention **invariante aux translations**, ce qui est essentiel en musique où la répétition et la variation jouent un rôle fondamental. Il augmente la capacité du modèle à reconnaître et reproduire des structures musicales, même lorsqu'elles sont transposées ou déplacées dans le temps.

L'intégration d'embeddings relatifs constitue donc un levier fondamental pour rendre le Transformer musicalement pertinent, généralisable et cohérent sur de longues séquences.

4.2 Etude Expérimentale

4.2.1 La programmation

Une fois cette structure pensée, nous sommes passés à la phase de programmation. Etant donné le court temps imparti et le fait que nous n'avions aucune formation préalable en machine learning et en programmation spécifique au machine learning nous avons été contraints de faire du prompt engineering, c'est-à-dire programmer main dans la main avec Chat GPT. Cette méthode, bien qu'elle semble être une voie de facilité est en fait très piégeuse.

Il nous est arrivé quelquefois de nous retrouver avec des parties de code dont on comprenait l'utilité mais pas les mécanismes internes. Alors un nouvel enjeu s'est présenté à nous, apprendre à déchiffrer des parties de code qui étaient devenues des boîtes noires à nos yeux. Tout cela s'est fait à l'aide des librairies que l'on retrouve en annexe. Le code complet final est disponible en annexe ici.

4.2.2 Les problèmes rencontrés

4.2.2.1 Les Dataset

Au cours de notre expérience, nous avons aussi pu prendre conscience de l'importance et la difficulté de trouver une dataset adaptée à nos besoins. Nous avons passé une grande partie du temps, que nous avions consacré à l'étude expérimentale, à parcourir de nombreux sites à la recherche d'une dataset de Jazz, sans trop de succès.

En réalité, la plupart des dataset qui sont en ligne sont téléchargées de façon illégale et se font très vite retirer en raison de droit d'auteurs. Trouver le matériel nécessaire à entraîner un modèle de génération de langage est très simple puisqu'il suffit de prendre n'importe quel article sur internet, tandis que dans la génération de musique, c'est un tout autre problème. D'autant plus qu'il nous arrivait d'en trouver quelques unes sous format mp3 mais malheureusement la qualité des convertisseurs mp3 → midi disponibles en ligne n'était pas suffisante et les morceaux devenaient méconnaissables après conversion.

Nous avons fini par trouver une dataset en format Midi de 1276 mélodies de piano, et c'est avec cette dataset que nous avons entraîné notre modèle.

4.2.2.2 Les erreurs de programmation lors des premiers jets

À noter que tout au long du stage, il nous arrivait rarement de juger nos résultats sur la base de l'audio qu'ils généraient. Nous nous penchions plutôt sur des valeurs numériques, comme la Negative Log-Likelihood (NLL), notion introduite dans la bibliographie.

Lors de nos premiers essais, la *loss* commençait systématiquement à 3 et descendait jusqu'à environ 1,75, moment où elle stagnait. Or, d'après la formule :

$$\mathcal{L} = -\log(P) = 1.75 \quad \Rightarrow \quad P = e^{-1.75} \approx 0.17 \tag{2}$$

ce qui signifie que notre modèle attribuait en moyenne une probabilité de 17 % au bon token dans la suite d'une séquence, ce qui est bien loin des 100 % d'un modèle parfait.

Nous avons alors réinspecté le code afin de comprendre cette marge d'erreur. Le problème venait du **masque d'attention**. Celui-ci empêchait les tokens de "voir" les futurs dans une séquence, en bloquant tout ce qui suit :

$$\text{Autorisé ssi} \quad j < i \tag{3}$$

Cela pose problème dans les accords. Par exemple, dans un encodage en série :

```
[C4, E4, G4]
```

le token `G4` pouvait "voir" `C4` et `E4`, mais l'inverse n'était pas vrai : les premières notes de l'accord étaient aveugles aux suivantes.

Pire encore, l'ordre d'encodage n'était pas forcément temporel. Certains tokens pouvaient observer des événements situés plus tard dans le temps, comme illustré ici :

$$\text{Encodage :} \quad [\texttt{note}_1(t = 0.8),\ \texttt{note}_2(t = 0.5)] \quad \Rightarrow \quad \texttt{note}_2 \rightarrow \texttt{note}_1$$

Ce qui est incorrect : un événement `note_2` ne devrait pas accéder à `note_1` si elle est dans le futur temporel mais dans le passé dans l'ordre d'encodage.

Alors nous avons introduit un masque temporel basé sur le *tick absolu* de chaque note :

$$\text{ABS}_i = \text{bar}_i \times 192 + \text{offset}_i \tag{4}$$

Le masque n'autorise l'attention que si :

$$\text{ABS}_j \leq \text{ABS}_i \tag{5}$$

Ce masque permet :

- l'influence mutuelle des notes jouées en même temps (comme les accords),
- le respect du temps musical réel plutôt que l'ordre d'encodage arbitraire,
- une meilleure généralisation du modèle.

Grâce à ce nouveau masque temporel, nous avons observé une baisse significative de la NLL et une meilleure cohérence musicale dans les séquences générées.

5 Résultats et Analyse

5.1 Résultat de l'entraînement

Une fois que notre code était finalisé, nous avons recommencé l'entraînement sur le serveur distant mis à disposition par le laboratoire de LISV. Notre code était programmé pour indiquer la loss toutes les 500 itérations. Voici le les résultats que nous avons obtenu :

Nombre d'itérations	Loss train	Loss Verification
500	2.52	1.55
1000	1.80	1.89
1500	1.65	1.77
2000	1.62	1.72
2500	1.67	1.55
3000	1.61	1.62
3500	1.60	1.59
4000	1.41	1.61

Au terme de quatre mille itérations d'apprentissage sur les 1 276 fichiers MIDI, notre modèle de 85 millions de paramètres affiche une dynamique typique d'une convergence saine, puis l'amorce d'un sur-apprentissage, plus communément appelé overfit. "Overfitter" c'est lorsque notre modèle se met à apprendre par coeur les données sur lesquelles il s'entraîne, et n'apprend plus du tout à généraliser. Ainsi quand il est de nouveau confronté à des données qu'il n'a jamais vues, il est désemparé car n'a pas suffisamment appris à généraliser. Ce phénomène est à éviter à tout prix.

La perte de validation passe d'abord de 2,44 à 1,59 en 3 500 pas : c'est une réduction d'environ 35 %, signe que le modèle a rapidement assimilé les régularités mélodiques et rythmiques du jeu de données. Jusqu'à ce point, l'écart entre la perte d'entraînement et celle de validation demeure inférieure à 0.1, l'architecture (8 couches, n_embd = 256) reste donc bien dimensionnée par rapport aux quelques cinq millions de tokens d'apprentissage, et aucun overfit n'est encore visible.

Après le dernier redémarrage de session (checkpoint à 2 000 itérations), la perte d'entraînement continue pourtant de décroître jusqu'à 1,41 tandis que celle de validation remonte à 1,61 : on observe l'apparition soudaine d'une différence de 0.2, premier indice que le réseau commence à overfit et recopier des séquences particulières plutôt qu'à modéliser leur distribution. En d'autres termes, le modèle est maintenant capable d'apprendre toutes les régularités du corpus, car sa capacité dépasse la complexité statistique des donnée.

A ce moment nous avions atteint les limites de notre database, elle était trop petite pour aller plus loin sans que notre modèle l'apprenne par cœur. Alors, nous sommes revenus au checkpoint de l'itération 3500 car c'est là que notre modèle avait les meilleures valeurs et nous avons utilisé ces valeurs pour la génération du morceau de musique.

5.2 Résultat de génération musicale

À l'issue de l'entraînement, nous avons généré plusieurs séquences MIDI à l'aide de notre modèle. Après conversion audio via des synthétiseurs, nous avons procédé à une évaluation qualitative des résultats. Bien que les productions soient encore éloignées d'un morceau de jazz authentique, certains aspects encourageants méritent d'être soulignés :

— Une cohérence rythmique satisfaisante, obtenue grâce à notre système d'encodage *bar + offset*

— Des choix de hauteurs (pitch) non aléatoires, montrant que le modèle tente de construire une ligne mélodique logique

Les points négatifs sont les suivants :

— Absence quasi-totale de dimension harmonique (accords, contrepoint)
— Incapacité à reproduire les structures formelles typiques du jazz
— Sonorité artificielle due aux limitations des synthétiseurs MIDI de base
— Problèmes de tempo avec un enchaînement beaucoup trop rapide pour du jazz

Dans ces conditions, la production de séquences musicales même élémentaires représente déjà un résultat significatif. Les particularités étranges observées révèlent surtout les limites actuelles du modèle plutôt qu'un échec fondamental de l'approche.

5.3 Perspectives d'amélioration

Plusieurs axes se dégagent pour progresser :

— Ajout d'un flux chord
— Ajouter une résolution temporelle adaptative, le modèle pourrait apprendre à changer de granularité en fonction de la complexité du passage
— Ajout d'un flux tempo
— Augmentation du dataset avec des partitions jazz annotées
— Utilisation de synthétiseurs plus réalistes pour l'évaluation

Enfin, nous avons réussi à obtenir un modèle qui apprend et produit de la musique intelligible en utilisant le transformer. Nous avons pu constater que les paramètres et la dataset peuvent avoir des grands changement sur le résultat. Nous avons commencé avec une architecture légère, entraînée sur un petit dataset, sans GPU personnel, dans un temps très limité. Cependant les premiers résultats confirment que notre approche avec le transformer peuvent générer des musiques spécifiques. Cela ouvre plein de perspectives pour la suite des améliorations de notre approche.

6 Conclusion

Ce stage a été une immersion intense dans un domaine complexe et en pleine mutation : la génération musicale par intelligence artificielle. Partis sans formation préalable en IA, en programmation avancée et sans savoir lire une partition de notes, nous avons dû apprendre en parallèle la théorie (fonctionnement des Transformers, tokenization musicale, attention multi-tête…) et la pratique (codage, debugging, gestion d'environnement distant).

Nous avons découvert que générer de la musique ne se résume pas à "faire jouer un modèle". Cela implique une réflexion profonde sur la structure temporelle de la musique, sur le format des données, sur les ressources limitées et surtout sur les limites actuelles des outils existants. Même les modèles commerciaux comme Suno AI ou MusicLM, pourtant entraînés sur des milliers d'heures de musique, échouent souvent à capturer l'esprit d'un style comme le jazz.

En partant d'un modèle brut, nous avons conçu une architecture complète de type Transformer multi-flux, entraîné sur une base MIDI, et testé différentes formes de tokenization. Les

résultats obtenus sont perfectibles, mais démontrent la possibilité de générer une séquence cohérente à partir d'un modèle que nous avons entièrement structuré.

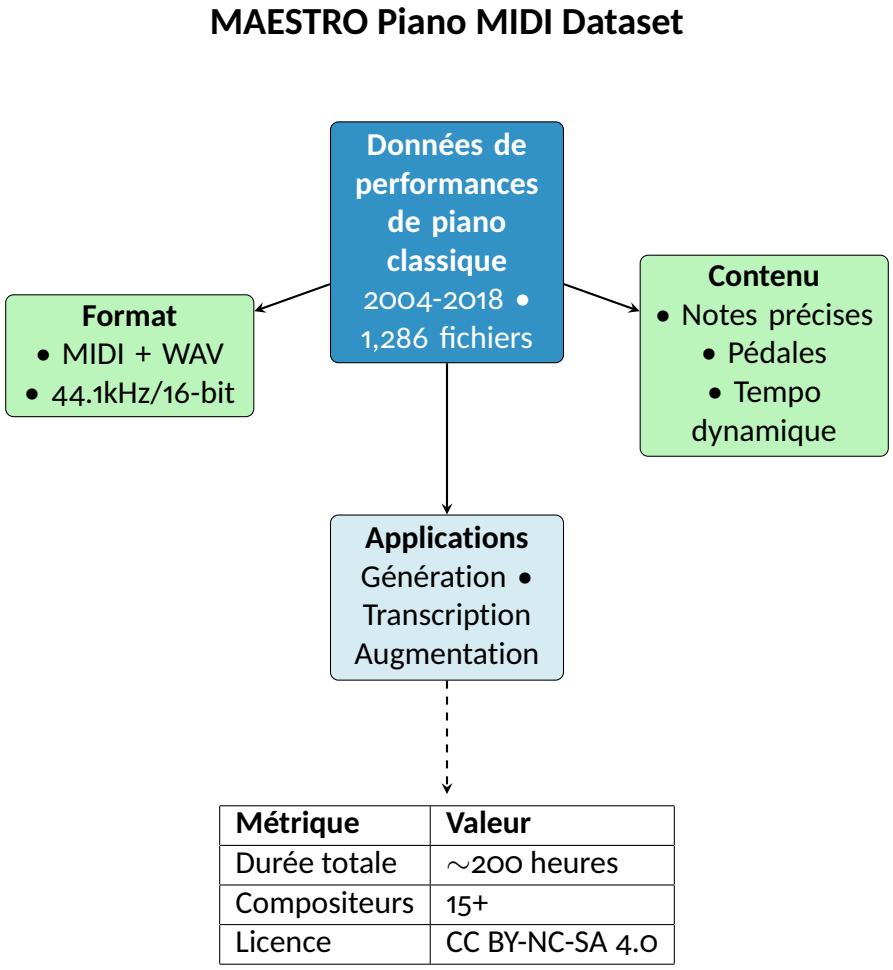
Ce stage nous a aussi sensibilisés aux questions culturelles et éthiques : le droit d'auteur, les "jumeaux vocaux" des artistes, ou encore le débat sur la créativité des machines. L'intelligence artificielle n'est pas encore capable de composer un morceau de jazz digne d'un musicien humain — mais elle peut déjà en esquisser l'ossature. L'enjeu n'est peut-être pas de remplacer l'artiste, mais de l'assister, de l'inspirer ou de lui proposer des pistes nouvelles.

En définitive, ce stage fut pour nous bien plus qu'un projet informatique. Il a été une expérience intellectuelle complète, à la fois technique, artistique et humaine, et il nous a donné envie d'en apprendre davantage sur l'IA, ses usages créatifs et ses futurs possibles.

7 Annexe

7.1 Dataset midi

Les données utilisés pour générer de la musique ont été prises depuis le site Kaggle sur la database au nom de Maestro-Piano-Midiaccrssible à ce lien https://www.kaggle.com/datasets/kritanjalijain/maestropianomidi, elle contient 1276 fichiers MIDI de performances de piano classique, synchronisés avec des enregistrements audio.



Source : kaggle.com/datasets/kritanjalijain/maestropianomidi

Figure 5 – Schéma synthétique du dataset MAESTRO Piano MIDI